

AutoTradeBot

Learning from its own trades - the machine-learning plan, in detail

A companion to the architecture guide · September 2026

What the model answers · the data and its three populations · the features · leakage and bias · validation · the model · how it plugs in · the timeline · what to read

1. What we are trying to learn, and what we are not

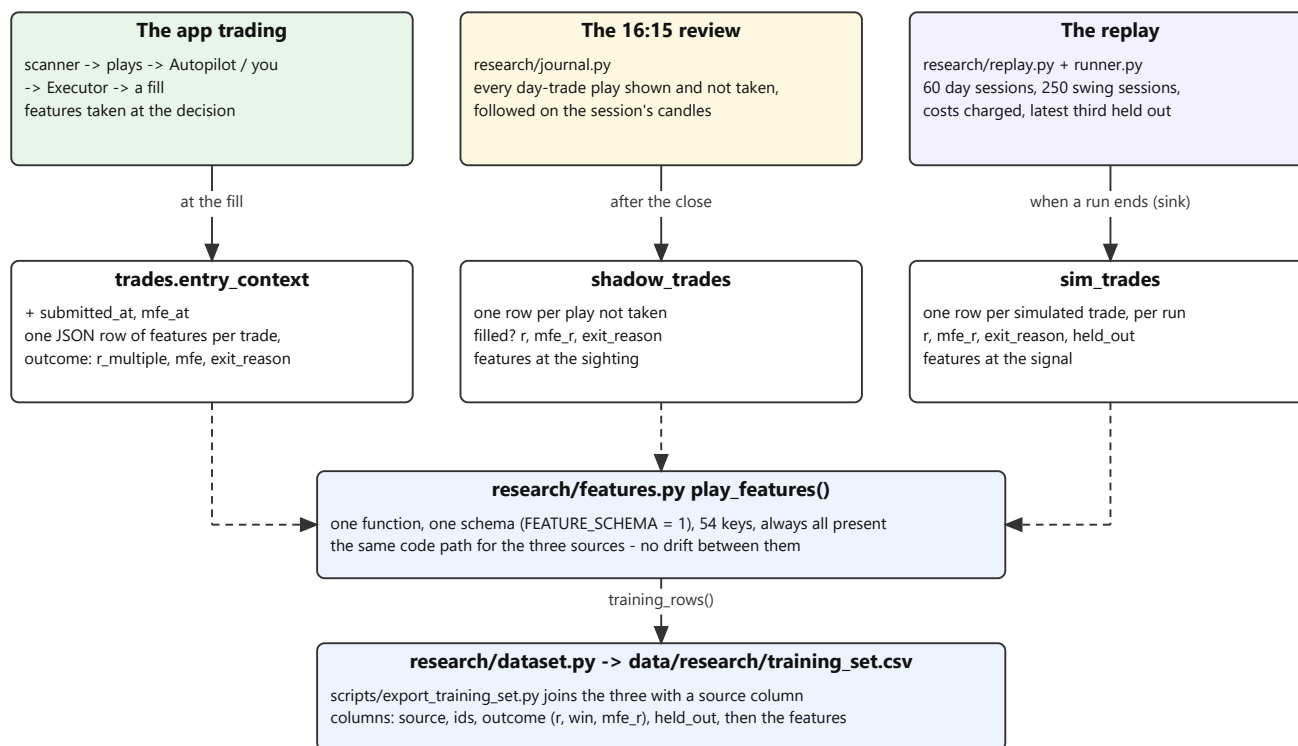
The one question. The app already has twenty hand-written setups that say *when* a trade is worth proposing. What it lacks is a calibrated answer to: *given this play, right now, how likely is it to reach its target before it hits its stop?* Today that number is a flat confidence the strategy states (0.55 to 0.62 for most setups), nudged by a pooled win rate. The plan is to replace it with a probability learned from what actually happened to thousands of plays like it. In the literature this is **meta-labelling** (López de Prado, ch. 3): the primary model decides the side and the moment, the secondary model decides whether to act and how much.

Why this and not a price predictor. Predicting prices from a few thousand rows is hopeless and the books you chose say so. Judging a proposed trade is a much easier problem: the label is binary, the features are the readings the setup already computes, and the base rate is around 40 to 55 percent, so even a modest lift in precision at the top of the ranking is worth money. It also keeps every decision explainable: the play panel can show the probability and the three features that moved it.

What the model never does. It never moves a stop or a target, never overrides the daily loss stop, the position caps or the pattern-day-trader rule, and never sizes a trade directly. It feeds two existing inputs: the gate's probability floor and the half-Kelly risk fraction.

Is it worth it? Only with enough rows and honest validation. Section 4 is the part that decides whether the model is used at all: if it cannot beat the hand-written confidence on purged walk-forward folds, it stays in shadow mode and the app is no worse than today.

2. The data: three populations, one schema



The app keeps running throughout: collection is passive, and the export reads the database without stopping anything.

Figure 1 - Where the training rows come from: the trades taken, the plays not taken, and the replay, all described by the same feature function.

Learning only from the trades the app took would teach it the gates' preferences, not the market: Autopilot refuses most plays, so the taken ones are a biased slice. Two more populations fix that.

Population	Rows	Label	Strength	Weakness
live	trades the app took, paper or live; features kept at the fill in <code>trades.entry_context</code>	the real R multiple, MFE, exit reason	real fills, real slippage, the only rows that count double in the evidence weights	few - tens per month - and biased by the gates
shadow	every day-trade play shown and not taken, followed by the 16:15 review on the session's 5-minute candles as if it had been taken; <code>shadow_trades</code>	simulated R with the exit manager's rules and the replay's costs	removes the selection bias: what the gates refused, and how it went	day trades only for now; swing plays not taken need days of candles (a follow-up)
replay	every simulated trade of a replay run, features at the signal; <code>sim_trades</code>	simulated R, with a <code>held_out</code> flag for the latest third of sessions	thousands of rows at once, across many stocks and regimes	no live readings in the replay (the volatility and character readings are None there); fills are idealised

How many rows to expect

Source	Today	After the first full-universe replay	After a month of paper trading
live	0 (older trades carry no context)	0	50 to 150

Source	Today	After the first full-universe replay	After a month of paper trading
shadow	0	0	roughly 20 to 60 per session, 400 to 1,200
replay	0 until the next run	about 9,000 (37 stocks gave 9,073); the full list gives more	the same, refreshed weekly

The first model will therefore rest on replay rows, be checked against shadow rows as they arrive, and be trusted only once the live rows agree with it.

3. The features

One function, `tos_bot/research/features.py play_features()`, turns a play into a flat dict of 54 keys. The same function runs at a live fill, in the review and inside the replay's worker processes, so the three sources cannot drift apart. Every key is always present (None when the reading was not available), and `FEATURE_SCHEMA` (now 1) goes up whenever a key is added or changes meaning. That is what makes the collection independent of everything else you change in the app: a row written last month is still readable and still labelled with the version it was written under.

The rule for a feature: it must have been knowable at the moment of the decision. Nothing that depends on later candles, on the fill, or on the outcome is allowed. The exit reason and the R multiple are labels, never inputs.

Key	Meaning	Comes from
schema	the layout version	features.py
strategy	which setup proposed the play	the play
kind	TECHNICAL or FUNDAMENTAL	the play
timeframe	INTRADAY (day trade) or SWING	the play
side	LONG or SHORT	the play
sector	the stock's sector (IBKR contract details)	the play
confidence	the setup's own conviction, 0-1, scaled by time of day	the strategy
probability	the odds the play stated (calibrated against the record)	strategies/base.py
reward_risk	(target - entry) / (entry - stop)	the play
score	the board rank: expected R x weight + bumps	scanner/filters.py
expected_r	odds x reward - (1 - odds), before costs	scanner/filters.py
evidence_weight	the strategy's multiplier from its record, 0.5-1.5	research/weights.py
stop_pct	distance to the stop, % of entry	geometry
target_pct	distance to the first target, % of entry	geometry
n_targets	1 or 2 (2 = scale-out at the first)	geometry
noise	the flags raised: against_trend, news_driven_move, turbulent_market ...	scanner/noise.py
n_noise	how many flags	scanner/noise.py
confirmations	scans in a row the setup showed	the board
tags	gap, catalyst, earnings ...	the strategy
has_catalyst	a gap or catalyst tag	the strategy
minutes_since_open	when in the session the play was seen	util/clock.py
time_of_day	OPEN, LATE_MORNING, MIDDAY, CLOSE, OFF (Aziz's sessions)	util/clock.py
weekday	0 = Monday	the clock
p_turbulent	probability the market is in its turbulent state	engine/market_regime.py
regime	calm or turbulent	engine/market_regime.py
rvol	volume vs the usual at this time	scanner/heat.py
gap_pct	today's open vs yesterday's close, %	scanner/heat.py
atr_pct	the daily average true range, % of price	scanner/heat.py
change_pct	today's move so far, %	scanner/heat.py
range_atr	today's range in daily ATRs	scanner/heat.py
heat	the attention score	scanner/heat.py

Key	Meaning	Comes from
dollar_volume	20-session average dollar volume (swing plays)	scanner/heat.py
move_atr	last session's move in ATRs (swing plays)	scanner/heat.py
extreme	0 mid-range .. 1 at a 20-session high or low (swing plays)	scanner/heat.py
vol	tomorrow's forecast daily volatility (GARCH)	quant/volatility.py
recent_vol	the last 60 sessions' volatility	quant/volatility.py
vol_ratio	forecast / recent - above 1 means rising	quant/volatility.py
vol_model	garch or ewma	quant/volatility.py
price_character	trending, mean reverting or random walk	quant/readings.py
hurst	the Hurst exponent	quant/stationarity.py
variance_ratio_z	the variance-ratio test statistic	quant/stationarity.py
half_life_bars	mean-reversion half-life, in bars	quant/stationarity.py
market_z	today's move vs what the market model expected, in sigmas	quant/market_model.py
market_beta	beta to SPY	quant/market_model.py
market_move_pct	today's move, %	quant/market_model.py
news_since_move	headlines since the move started	signals
sessions_to_earnings	sessions until the next report	signals/calendar.py
signal_nudge	the score nudge from insider / news signals	signals/book.py
replay_expectancy_r	the strategy's replayed average R when the trade was taken	live trades only
replay_trades	how many replayed trades that rests on	live trades only
held_out_r	the held-out average R	live trades only
risk_pct	the risk % it was sized at (half-Kelly)	live trades only
unproven	whether the proof rule would have refused it	live trades only
by	autopilot or operator	live trades only

The labels

- **r**: the outcome in risk units (R). +2 means the trade made twice what it risked; -1 means it hit its stop.
- **win**: $r > 0$. The classification target.
- **mfe_r**: the best the trade got before it closed, in R. Useful as a softer target ("did it ever reach +1R?") and for judging the exit rules separately from the entries.
- **exit_reason**: stop, target, trailing-stop, eod-flatten, time-stop. Lets us ask whether a setup loses because it is wrong or because it runs out of time.

4. Leakage, bias and honest validation

This is the section that decides whether the model is used. Every trap below has caught published strategies; Aronson and López de Prado each spend half a book on them.

1. **Look-ahead leakage.** A feature computed with candles the decision could not have seen. Guarded by construction: the replay builds the context bar by bar and the live path takes the features at the fill. The test suite checks the schema, but a new feature must be reviewed for this by hand.
2. **Selection bias.** Training only on taken trades. Guarded by the shadow rows (section 2).
3. **Overlapping labels.** Two rows from the same stock and session are not independent: a random train/test split leaks. Guarded by walk-forward folds with a purge gap (figure 2).
4. **Multiple testing.** Twenty setups, dozens of features and several models means something will look good by chance. Guarded by the shuffled-label baseline and by reporting how many configurations were tried (the deflated Sharpe idea).
5. **Regime drift.** A model fitted on a calm year fails in a turbulent one. Guarded by the regime feature, by folds that span both, and by monitoring live outcomes against predictions after deployment.

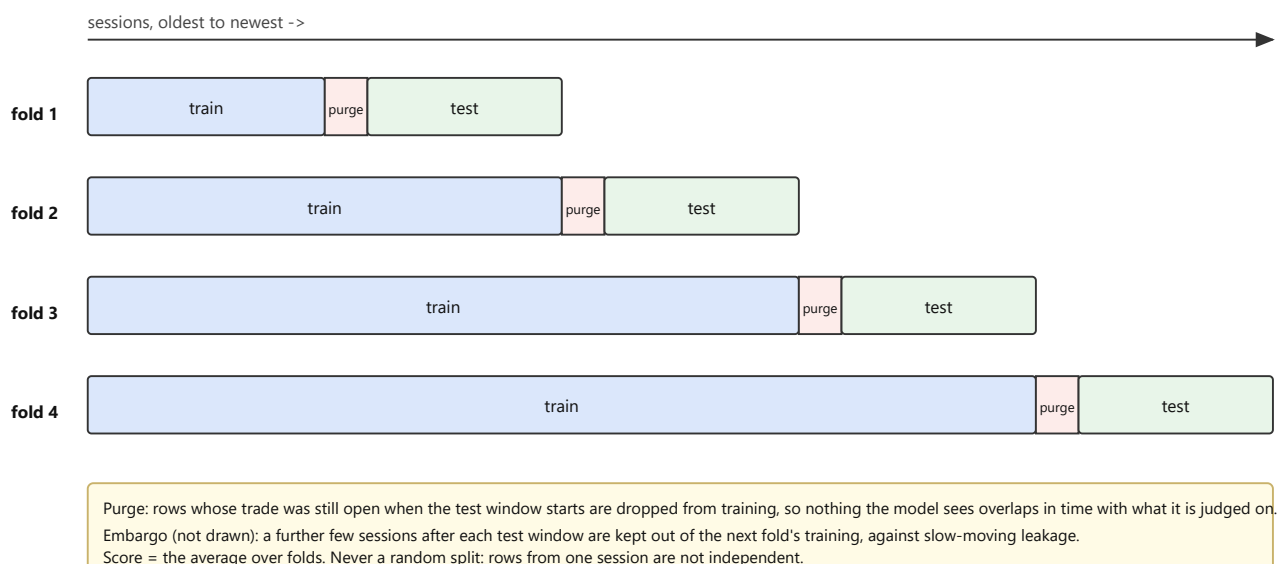


Figure 2 - Purged walk-forward validation: each fold trains on the past only, skips a purge gap, and is judged on the sessions after it (Lopez de Prado, ch. 7).

What gets measured, per fold

Metric	Question it answers	Pass mark
log loss and Brier score	are the probabilities honest?	better than the hand-written confidence on the same rows
calibration curve	does a 0.6 mean 60%?	within 5 points across deciles
precision at the top decile	if we only took the plays the model likes most, how often do they win?	clearly above the base rate
expectancy of the plays above the floor	in R, the number that matters	above the expectancy of the plays the current gates would take
shuffled-label baseline	how good does a model look on random labels?	the real model must beat the best of 100 shuffles
held-out replay sessions	the same test the strategies pass	the gain survives out of sample

5. The model

Start simple. A logistic regression on standardised features is the first model, because it is transparent, hard to overfit on a few thousand rows, and gives a sign for every feature that a trader can argue with. If it cannot beat the hand-written confidence, nothing fancier will help; the data is the problem, not the model.

Then trees. Gradient-boosted trees (scikit-learn's `HistGradientBoostingClassifier`; no GPU, no new heavy dependency) capture interactions such as "a Bollinger fade works at midday in a calm regime but not in the first hour". Use monotone constraints where the books are clear (more confirmations should not lower the probability), small depth, and early stopping on the walk-forward folds.

Calibrate. Whatever the model, its raw scores are passed through isotonic or Platt calibration fitted on the folds, so the number shown is a probability and the half-Kelly formula can use it.

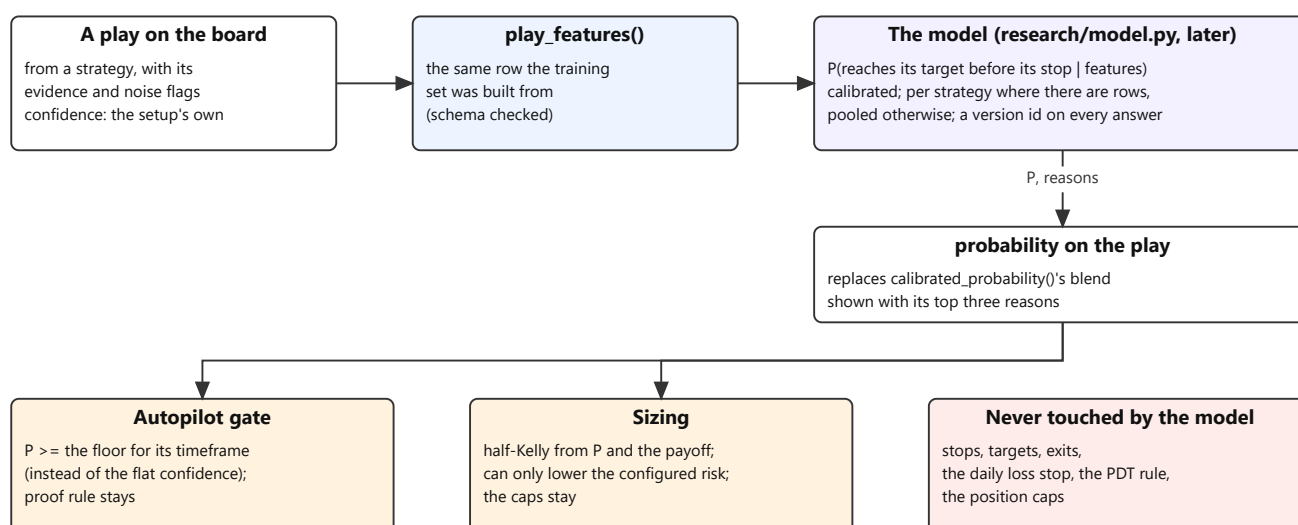
Pooled or per strategy. One pooled model with `strategy` as a feature, plus per-strategy models only where a strategy has more than about 300 rows. The pooled one is the fallback.

Explain every answer. For each play, the three features that moved the probability most, in words ("relative volume 3.1x: +0.08", "against the trend: -0.11"). Logistic coefficients give this directly; for trees, per-row contributions (SHAP-style) are cheap at this size.

Version everything. A model file carries the feature schema it was trained on, the rows it saw (counts per source, first and last session), the fold scores and a model id. The play panel and the journal show the id, so a bad week can be traced to the model that made it.

Retrain nightly. The engine retrains in the background after each day's 16:15 review has added its rows (`ResearchOps.train_model`); `scripts/train_model.py` does the same by hand. The scorer picks a new model up within a minute, without a restart.

6. How it plugs into the app

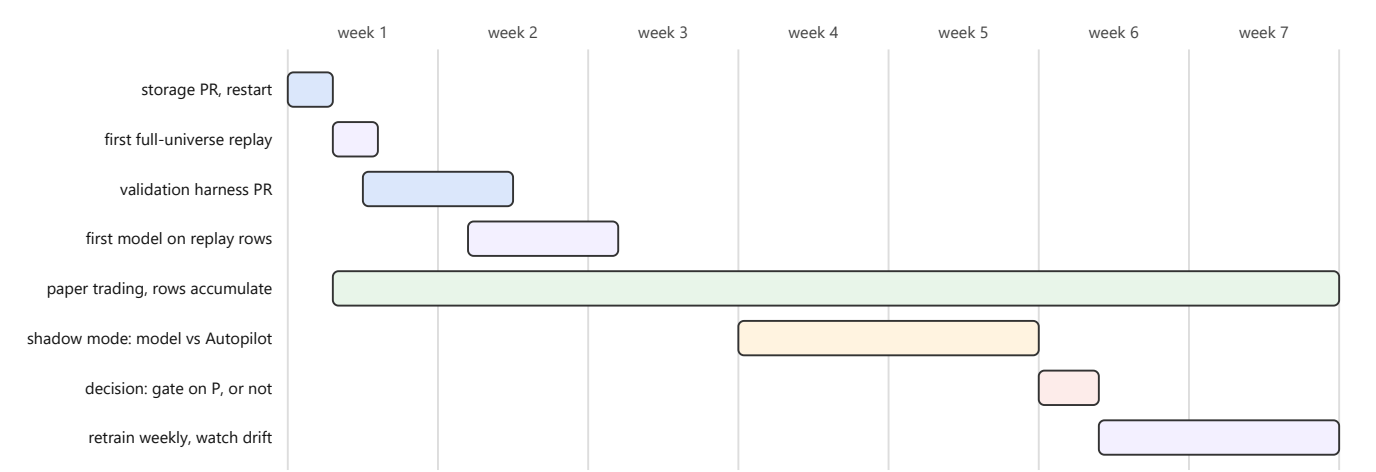


First two weeks in shadow: the model's verdict is logged next to Autopilot's ('would enter' / 'would skip') and nothing changes until the two are compared.

Figure 3 - Where the model's probability goes: the gate and the size, never the exits.

1. **Shadow mode first.** The model scores every play on the board and the verdict is logged next to Autopilot's. Nothing changes for two weeks. The journal reports where the two disagree and who was right.
2. **Then the probability.** The play's `probability` field comes from the model instead of the pooled blend; the board, the expected R, the rank score and the sizing all read it already.
3. **Then the gate.** Autopilot's confidence floor becomes a probability floor per timeframe. The proof rule stays: a strategy still has to earn its place in the replay.
4. **Sizing follows.** Half-Kelly already takes the win rate and the payoff; it now takes the play's own probability, still capped by the configured risk and the exposure ceilings.
5. **A kill switch.** One setting turns the model off and restores the hand-written confidence; the journal watches the live win rate against the predicted one and warns when they diverge.

7. The timeline and what "done" means



The long pole is the paper trading: real fills are the test Chan asks for, and fifty to a hundred of them take weeks.

Figure 4 - The timeline: about a week to a first model judged on replay rows, four to six weeks to one that includes real paper fills.

Milestone	Deliverable	Acceptance
Storage (this week)	the tables and the export	rows appear for all three sources after one session and one replay
Validation harness	research/validate.py : purged folds, baselines, the metric table	reproduces the hand-written confidence's numbers as the baseline
First model	research/model.py , a logistic model trained on replay rows	beats the baseline on log loss and top-decile precision across folds
Shadow mode	verdicts logged next to Autopilot's for two weeks	agreement report in the journal
Go / no-go	gate on the probability, or stay in shadow	the live rows agree with the folds
Operations	weekly retrain, drift watch, model ids in the journal	runs unattended

Can the rest of the app keep changing meanwhile? Yes. The rows are versioned by the feature schema and the model checks the schema it was trained on. New setups simply produce new rows; a changed setup should bump its own record (its replay rows are re-created on the next run anyway).

8. Where it stands (2026-09-18)

Everything in the plan up to shadow mode is built. What the numbers say so far is sobering, and that is the point of measuring:

Piece	What is built	What it says today
Records judged for luck	<code>research/significance.py</code> : Aronson's bootstrap and White's reality check across every setup tried, net of each stock's own drift; Tharp's quality number and marble-bag drawdowns; Carver's cost share	no setup is proven: the best one's +0.10R has a 29% chance of being luck alone and 99% once the other setups tried are allowed for
The meta-label model	<code>research/model.py</code> : boosted trees, uniqueness weights with time decay, purged walk-forward verdict, isotonic calibration, out-of-sample feature importance; retrained nightly; scores every play	not usable on 4,868 rows: it ties the stated odds on log loss and its top decile wins no more than the base rate. It runs in shadow and has no say
Execution quality	the quote at the decision is stored with every trade, slippage is measured at the fill, a wide spread refuses the entry on live quotes, stale day-trade entries are cancelled	no live fills measured yet; the daily review reports the averages against the replay's 6 bps a side once there are five
Three more book setups	<code>strategies/patterns.py</code> : Grimes's failure test and pullback, Bulkowski's confirmed double bottom	a year's replay on 400 stocks: -0.01R to -0.09R after costs, shorts worse than longs. They face the same proof as every other setup
An independent audit	<code>scripts/r/audit_records.R</code> recomputes the record statistics in base R	it agrees with the Python numbers to the second decimal

What would change the picture. Rows and better features, not a cleverer model: every day adds shadow rows at no risk; real-time data makes the day-trade features current instead of 15 minutes old; and the features with the highest information coefficients so far (recent volatility, the stop distance, the market's move) suggest the next ones to build. The honest default until then: Autopilot's proof rule keeps it out of unproven setups, and paper practice with the rule switched off is for collecting rows, not for expecting profit.

9. What would help most, and what to read

Data before books. The single most useful thing is to run the replay after each morning's full scan and to keep the app paper trading with the proof rule on. Rows are the constraint.

Books and papers, in the order they will be used:

Title	Why it matters here	Used in
Advances in Financial Machine Learning , Marcos López de Prado (Wiley, 2018)	meta-labelling (ch. 3), purged cross-validation and embargo (ch. 7), backtest overfitting and the deflated Sharpe ratio (ch. 11-14), feature importance done honestly (ch. 8)	sections 1, 4, 5
Evidence-Based Technical Analysis , David Aronson (Wiley, 2006)	data-mining bias in chart patterns; how to test a rule and how many rules you can test before the best one is noise	section 4; deciding which of the 13 technical setups to keep
The Probability of Backtest Overfitting , Bailey, Borwein, López de Prado and Zhu (Journal of Computational Finance, 2017)	the CSCV method: how likely it is that the best configuration found in-sample underperforms out of sample	section 4's multiple-testing check
Meta-labeling articles by Hudson & Thames (online, 2019-2021)	worked examples of the secondary-model idea with code	section 5
Trading and Exchanges , Larry Harris (Oxford, 2003)	market microstructure: why fills slip, how limit orders queue, what the open does; turns the replay's 5 bps slippage into a modelled number	the replay's costs; the order builder
Machine Trading , Ernest Chan (Wiley, 2017)	the third Chan book: practical ML for strategies, with the same scepticism as the first two	background

Only the first two are needed to build what this guide describes. The rest sharpen it.

Glossary

Term	Meaning
Meta-labelling	a second model that decides whether to act on a first model's signal, and how much
Calibration	a predicted 0.6 should come true about 60% of the time
Purged walk-forward	train on the past, test on the future, drop the rows that overlap the boundary
Embargo	extra sessions after a test window kept out of the next training set
Shuffled-label baseline	train on random labels to see how good 'nothing' looks
Deflated Sharpe / PBO	corrections for having tried many configurations before picking the best
Base rate	the win rate before any model: what you must beat
Log loss / Brier score	how far predicted probabilities are from what happened, penalising confident mistakes
Half-Kelly	half the bet fraction that maximises log growth, from win probability and payoff
Shadow mode	the model runs and is logged, but nothing acts on it
Drift	the world changing under a fitted model; watched by comparing live outcomes with predictions

Generated from the code on the learning-storage branch, 16 September 2026.